

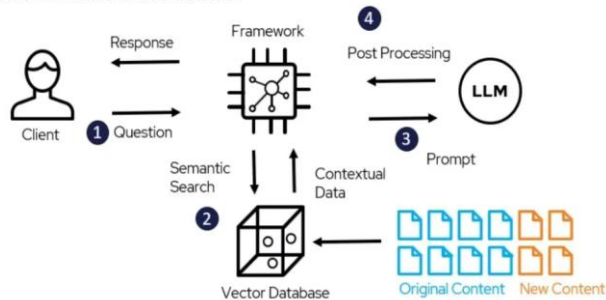
GenAI Training Day 3

Retrieval-Augmented Generation (RAG)

Overview

- RAG enables knowledge-grounded LLM responses
- Connects models to private, real-time, and factual data
- Reduces hallucinations by grounding generation in retrieved text

RAG Architecture Model



The Core Problem with LLMs

- No access to private or real-time data
- No long-term memory
- Generate plausible text, not verified truth
- Can hallucinate confidently

What RAG Is (and Is Not)

- RAG = Retrieve first, then generate
- Grounds responses in external knowledge
- Not fine-tuning
- Not built-in memory
- Not Google search

Key Components of RAG

- Generation model (LLM)
- Embedding model
- Retriever
- Vector database
- Prompt template
- Chunker (Use Case Dependent)

Why Embeddings Power RAG

- Convert text into numerical vectors
- Capture semantic meaning, not exact words
- Similar meaning = closer vectors
- Used for retrieval, not generation

Vector Databases

- Store embeddings + metadata
- Enable fast similarity search

- Act as internal knowledge bases
- Examples: FAISS, Pinecone, Chroma, Weaviate, Milvus

Why Not Traditional Databases?

- SQL = exact keyword matching
- Semantic meaning $\neq$ keywords
- Vector search retrieves by similarity

Types of searches

- **Keyword search:** exact or fuzzy matches
- **Semantic search:** meaning-based retrieval
- **Vector search:** similarity via embeddings
- **Hybrid search:** combines keyword + vector

Retrieval & Ranking

- Retriever scores document relevance
- Top-K documents selected
- Re-ranking improves result ordering
- Better ranking = better generation

Why RAG Matters

- Significantly reduces hallucinations
- Enables enterprise and private AI
- Connects LLMs to real-world data

Best Practices

- Chunk documents properly
- Use metadata filtering
- Tune Top-K retrieval
- Apply re-ranking
- Cite sources when possible